

Questão 1

Qual é a saída? O problema é detectado na compilação, na execução ou nunca?

```
console.log(0.1 * 3 === 0.3);  
console.log(9007199254740993);
```

Resposta: A saída para o primeiro console.log é false pois 0.1 não é convertido para binário de forma exata, então não resulta em exatamente 0.3. Já o segundo console.log não apresenta exatamente o mesmo número colocado pois ele é arredondado para baixo, também devido a não conseguir representar os bits exatos desse número. O problema nunca é detectado, pois arredondar esses números é o comportamento esperado.

Questão 2

Qual é a saída? O problema é detectado na compilação, na execução ou nunca?

```
palavra = "maçã"  
print(len(palavra), len(palavra.encode())) # encode() usa UTF-8 por padrão
```

Resposta: a saída é 4 e 6, pois o primeiro length calcula o número de caracteres na string, e maçã tem exatamente 4 letras. No segundo length, o encode converte a string para bytes, então os caracteres especiais de ç e ã contam como 2 bytes, resultando em 6. Nenhum tipo de erro ocorre no programa.

Questão 3

Qual é a saída? O problema é detectado na compilação, na execução ou nunca?

```
var b byte = 255  
b++  
fmt.Println(b)
```

Resposta: a saída é 0 pois o número alocado para o byte inicialmente já está no limite, e ao iterar ele a memória é estourada. Esse estouro nunca é detectado, pois não é um erro.

Questão 4

Qual é a saída? O problema é detectado na compilação, na execução ou nunca?

```
int[] v = new int[3];  
System.out.println(v[0]);  
System.out.println(v[3]);
```

Resposta: a saída do primeiro println é 0, pois é o valor padrão para os elementos de um vetor quando criado e nada é inserido. No segundo println ocorre um erro de execução que interrompe o programa, pois não existe a posição 3 no vetor criado, e sim apenas a 0, 1 e 2, passando então do limite.

Questão 5

Qual é a saída? O problema é detectado na compilação, na execução ou nunca?

```
let s = String::from("oi");  
let t = s;  
println!("{}", s, t);
```

Resposta: o código não compila, com o problema sendo detectado na compilação, pois ao passar o valor de s para t, em rust, o que havia na memória da variável s é transferido pra t, então ao tentar imprimir ambos, não há nada alocado em s mais, gerando assim o erro.

Questão 6

Qual é a saída? O problema é detectado na compilação, na execução ou nunca?

```
union { int i; float f; } u;  
u.f = 1.0f;  
printf("%d\n", u.i);
```

Resposta: a saída é 1065353216, pois o 1.0 em ponto flutuante é representado pelo computador na sequência de bits como se fosse um inteiro, resultando no valor apresentado. Não há nenhum problema detectado no código.